However, there is an implicit conversion between an `int` and an `EventId`, so you are more likely to use the following approach:

```
_logger.LogWarning(404, "City \"{name}\" not found", Name);
```

Event IDs are not required as part of a log. If you don’t provide a value, the event ID defaults to zero.

12.3.5 Formatting the log message

The string you pass to the various `Log*` methods is referred to as a *template*. It can contain placeholders for variable values, which are supplied as a `params` array argument. Your examples so far include only one parameter, `{name}`, representing the city name, the value for which is passed in after the template. When constructing a template, you are advised to use named placeholders in this way. This is primarily so you can take advantage of the benefits of *structured logging*. Some third-party logging libraries are able to store the parameter names and values separately, which makes it easier to query logs for more detailed analysis. This is known as *structured*, or *semantic*, logging.

The log message template looks similar to a standard format string, but it does not behave in the same way. Parameter values are inserted into the template according to their position in the parameter array, not according to their name. You must provide at least as many arguments to the parameter array as there are placeholders in the template; otherwise, a runtime exception occurs. This means if you want to repeat a value within the message template, you must include it in the parameter array again, like this:

```
_logger.LogWarning("{name} searched for. {name} not found", Name, Name);
```